UPDATE 8efa005

Sumário

Guia de Estudo e Documento de Síntese: Atualizações do Sistema Wiki-alternative (Commit 8efa005) 2
Relatório de Síntese 2
Resumo Executivo 2
Análise Detalhada dos Temas Principais 3
Questionário de Avaliação (Quiz) 5
Questões de Resposta Curta 5
Chave de Respostas do Questionário 5
Perguntas de Desenvolvimento (Ensaio) 6
Glossário de Termos-Chave 7
Relatório Técnico de Arquitetura e Integração de Inteligência Artificial — Versão 4.1.0 8
1. Módulo de Sanitização e Limpeza de Texto em Documentos 8
1.1. Importância Estratégica e Conceitos de Sanitização 8
1.2. Mecanismos Algorítmicos de Limpeza e Preservação da Estrutura 8
1.3. Suíte de Testes Unitários e Validação em Linha de Comando (CLI) 9
2. Motor de Verificação de Integridade Académica e Antiplágio 10
2.1. Regras Normativas ABNT e Identificação de Inconformidades 10
2.2. Integração do Modelo Gemini 3.8-Flash e Grounding de Pesquisa Web 11
2.3. Tolerância a Falhas: Motor Heurístico Determinístico de *Fallback* 12
3. Guia Pedagógico de Otimização e Grounding para Programadores 13
3.1. Otimização de Desempenho no Frontend com Code-Splitting (React.lazy e Suspense) 13
3.2. Mecânica de Consulta com Grounding Web no Modelo Gemini 14
4. Documentação Técnica das Alterações no Commit 8efa005 (Versão 4.1.0) 15
4.1. Atualizações de Dependências, Scripts e Ficheiros de Configuração 15
4.2. Mapeamento Geral e Análise Ficheiro a Ficheiro 15
4.3. Avaliação do Impacto Arquitetural para Engenharia e Auditoria de Software 17

UPDATE 8efa005



Guia de Estudo e Documento de Síntese: Atualizações do Sistema Wiki-alternative (Commit 8efa005)

Relatório de Síntese

Resumo Executivo

O *commit* `8efa005` do repositório `WazzimaGiygg/Wiki-alternative` marca uma evolução estrutural significativa no sistema, com o incremento da versão de `0.0.0` para `4.1.0`. A atualização foca-se no fortalecimento das capacidades de Inteligência Artificial para análise de integridade académica e deteção de plágio, na otimização de desempenho e na introdução de ferramentas avançadas de tratamento e teste de texto.

Os três pilares fundamentais desta atualização consistem em:

1. **Atualização do Modelo de IA e Pesquisa Cruzada (*Grounding*):** Migração do modelo base de IA para `gemini-3.8-flash`, acompanhado pela integração de pesquisas na Web em tempo real (*Google Search grounding*) e um motor de *fallback* heurístico que cruza dados com repositórios académicos consagrados (SciELO, USP, CAPES e Google Scholar).
2. **Robustez no Tratamento de Documentos e OCR:** Introdução do suporte a Reconhecimento Ótico de Caracteres (OCR) via biblioteca `tesseract.js` (v7.0.0) e criação do módulo `textSanitizer.ts`, projetado para higienizar ficheiros de texto

(PDF, DOCX, DOC, TXT), eliminando artefactos binários, quebras anómalas e mantendo a conformidade com as normas ABNT.

3. **Otimização de Desempenho e Qualidade de Código:** Implementação de divisão de código (*code-splitting*) a pedido no componente `AntiplagiarismTccView` recorrendo a `React.lazy` e `React.Suspense`, além da inclusão de uma suíte executável de testes unitários CLI contendo 10 casos de teste canónicos para validação do sanitizador de texto.

Análise Detalhada dos Temas Principais

1. Evolução do Motor de IA e Mecanismos de *Grounding* Académico

A arquitetura do servidor (`server.ts`) e dos serviços de antiplágio foi reformulada para garantir maior precisão e resiliência nas análises de trabalhos de conclusão de curso (TCC):

- **Migração do Modelo:** O motor primário de IA foi atualizado de `gemini-2.5-flash` para `gemini-3.8-flash`, aumentando o tempo limite de execução (*timeout*) de 25 000 ms para 30 000 ms.
- **Instruções de Sistema Especializadas:** A chamada de IA inclui agora uma instrução de sistema que define o modelo como um avaliador de integridade académica, plágio e normas ABNT (especificamente as normas NBR 10520 para citações e NBR 6023 para referências bibliográficas).
- **Análise de *Grounding* na Web:** A integração com a ferramenta `{ googleSearch: {} }` permite extrair metadados de suporte (`groundingMetadata.groundingChunks`). O sistema analisa os domínios encontrados e categoriza as fontes em:
  - `academic_paper`: Domínios contendo `.edu`, `.gov.br` ou `scielo`.
  - `encyclopedia`: Domínios associados a `wikipedia` ou `wiki`.
  - `web_search`: Outras fontes gerais da Web.
- **Motor Heurístico Determinístico de *Fallback*:** Na eventualidade de indisponibilidade ou erro de análise da IA Gemini, o servidor ativa um motor de análise heurística. Este motor simula cruzamentos com bases de dados científicas através de URLs de pesquisa dinâmicos baseados no título e palavras-chave do documento:
  - **SciELO Brasil:** Artigos e modelos em revistas científicas.
  - **Biblioteca Digital de Teses e Dissertações da USP (`teses.usp.br`):** Repositório de mestrados e doutoramentos.
  - **Google Académico (`scholar.google.com.br`):** Citações e métricas de impacto.
  - **Portal de Periódicos CAPES / MEC (`capes.gov.br`):** Publicações de alto impacto nacional (Qualis).

Motor Utilizado	Identificador (`usedEngine`)	Condição de Ativação
Gemini com Grounding	`gemini_search_grounding`	Resposta com sucesso da IA Gemini contendo fontes web válidas.
Gemini IA Simples	`gemini_ai`	Resposta da IA Gemini sem metadados de pesquisa web.
Sintaxe Heurística	`heuristic_syntax`	*Fallback* ativado em caso de falha da API ou erro de *parse* de JSON.

2. Higienização de Texto e Validação com Testes Automatizados (`textSanitizer.ts`)

UPDATE 8efa005

A extração de texto proveniente de PDFs, ficheiros do Word ou digitalizações é frequentemente acompanhada por ruído estrutural. O módulo `src/utils/textSanitizer.ts` resolve esta limitação através da função `cleanExtractedText()`, que aplica um pipeline de limpeza em 8 etapas:

1. **Eliminação de Caracteres de Controlo:** Remove bytes nulos (`\x00`) e caracteres não imprimíveis (`\x01-\x08`, `\x0B`, `\x0C`, `\x0E-\x1F`, `\x7F-\x9F`), preservando explicitamente tabulações (`\t`) e quebras de linha (`\n`).
2. **Limpeza de Caracteres Invisíveis:** Remove hífens suaves (`\u00AD`, convertidos em -) e caracteres de largura zero (`\u200B-\u200D`, `\uFEFF`).
3. **Normalização de Terminações de Linha:** Padroniza quebras de linha Windows (`\r\n`) e Mac antigo (`\r`) para o padrão Unix (`\n`).
4. **Remoção de Artefactos de Estrutura PDF:** Filtra marcas de cabeçalho `%PDF-`, definições de objetos (`obj...endobj`), blocos binários (`stream...endstream`), tabelas de referência cruzada (`xref...trailer`) e marcadores de fim de ficheiro (`%%EOF`).
5. **Descarte de Paginação Separada:** Remove números de página isolados situados entre parágrafos (ex.: marcas como `\n\n12\n\n` ou `Página 12 de 45`).
6. **Recomposição de Hifenização de Fim de Linha:** Utiliza a expressão regular `([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])` para unificar palavras divididas entre linhas (ex.: transformando "sustenta-\n bilidade" em "sustentabilidade").
7. **Normalização de Espaços Horizontais:** Reduz múltiplos espaços em branco, tabulações e espaços insecáveis (`\u00A0`) a um único espaço simples.
8. **Preservação de Estrutura ABNT:** Mantém a integridade de parágrafos separados por duas quebras de linha (`\n\n`) e preserva na íntegra a pontuação, aspas e parênteses de citações diretas.

Para garantir a fiabilidade contínua do sanitizador, o projeto inclui um banco canónico de 10 casos de teste unitários executáveis na linha de comandos via `scripts/test-sanitization.ts` (`npm run test:sanitization`):

```
1. test-null-bytes (control_chars)
2. test-windows-line-endings (line_endings)
3. test-mac-line-endings (line_endings)
4. test-pdf-header-artifacts (pdf_artifacts)
5. test-hyphenation-simple (hyphenation)
6. test-hyphenation-with-spaces (hyphenation)
7. test-soft-hyphens (hyphenation)
8. test-isolated-page-numbers (pdf_artifacts)
9. test-whitespace-and-tabs (whitespace)
10. test-abnt-citations-preservation (abnt_quotes)
```

3. Arquitetura do Front-end e Ajustes na Interface de Utilizador

- **Otimização de Pacotes (*Code-Splitting*):** Em `src/App.tsx`, o carregamento do módulo pesado de análise de plágio (`AntiplagiarismTccView`) foi alterado para ser feito a pedido (*lazy loading*). O componente é importado utilizando `React.lazy()` e envolvido por um bloco `React.Suspense`. Caso o módulo esteja a ser descarregado, é exibida uma interface de carregamento com *spinner* animado e texto indicativo (`4.1.1.222.6 Code-Splitting`).
- **Integração de OCR:** A dependência `tesseract.js` (versão `^7.0.0`) foi adicionada ao `package.json` para permitir o processamento de imagens e documentos digitalizados.

- **Modelos de Dados TypeScript:** A interface `ParagraphAnalysisItem` em `src/types/antiplagiarism.ts` passou a suportar a propriedade opcional `matchedSources?: MatchedWebSource[]`, e o relatório geral (`AntiplagiarismReport`) inclui agora contadores de páginas processadas por OCR (`ocrProcessedPages`) e a listagem completa de fontes de *grounding* capturadas na Web (`groundingWebSources`).
- **Atualizações de Identificação de Versão:** Os componentes visuais `Header.tsx` e `SiteUpdatesView.tsx` foram modificados para apresentar explicitamente a marcação de versão `Google V4.1` e `WikiWorldWeb V4.1`.

Questionário de Avaliação (Quiz)

Questões de Resposta Curta

1. Qual é o modelo de IA que passou a ser utilizado no *commit* `8efa005` e qual o tempo limite (*timeout*) atribuído às suas requisições?
2. Como funciona a extração e classificação de metadados de *grounding* realizada durante a chamada da API do Gemini?
3. Em que consiste o mecanismo de *fallback* heurístico do servidor e quais são os repositórios académicos simulados?
4. O que é o *code-splitting* implementado no ficheiro `src/App.tsx` para a vista `AntiplagiarismTccView` e qual o seu benefício?
5. Qual é a utilidade da inclusão da biblioteca `tesseract.js` no ficheiro `package.json`?
6. Quais os tipos de artefactos de ficheiros PDF que a função `cleanExtractedText` remove do texto bruto?
7. Como é efetuada a recomposição de palavras divididas por hifenização no final da linha dentro do sanitizador de texto?
8. Qual é a função do script `scripts/test-sanitization.ts` e de que forma ele sinaliza a falha em testes unitários?
9. Quais são as duas normas ABNT mencionadas explicitamente nas instruções de sistema do avaliador de integridade académica?
10. Quais as novas propriedades introduzidas nas interfaces de dados TypeScript do ficheiro `src/types/antiplagiarism.ts`?

Chave de Respostas do Questionário

1. **Resposta:** O modelo de IA foi atualizado para o `gemini-3.8-flash`. O tempo limite de execução das requisições na função `generateWithFallback` foi expandido para 30 000 ms (30 segundos).
2. **Resposta:** O sistema extrai a estrutura `groundingMetadata` dos candidatos da resposta da IA e percorre os fragmentos da Web (`groundingChunks`). Cada fonte é categorizada como `academic_paper` (domínios .edu, .gov.br ou SciELO), `encyclopedia` (Wikipédia) ou `web_search` (outros sites gerais).
3. **Resposta:** O *fallback* é uma análise determinística ativada em caso de erro na IA, que insere automaticamente sugestões de verificação cruzada baseadas no título do TCC. As bases simuladas incluem a SciELO Brasil, a Biblioteca Digital de Teses e Dissertações da USP, o Google Académico e o Portal de Periódicos CAPES.
4. **Resposta:** Trata-se do carregamento sob demanda do componente através de `React.lazy()` e `React.Suspense`. Esta técnica impede que o código do analisador de

antiplágio seja descarregado no carregamento inicial da aplicação, exibindo um *spinner* temporário enquanto a vista é obtida.

5. **Resposta:** A biblioteca `tesseract.js` (versão `^7.0.0`) foi adicionada para fornecer capacidades de Reconhecimento Ótico de Caracteres (OCR). Isto permite extrair texto a partir de imagens ou ficheiros digitalizados antes de os submeter à análise.
6. **Resposta:** A função elimina tags de versão (como `%PDF-1.7`), dicionários de catálogo (`<< ... >>`), declarações de objetos (`obj...endobj`), blocos de dados binários (`stream...endstream`) e secções de tabelas de referência (`xref/trailer`).
7. **Resposta:** Utiliza uma expressão regular que deteta um hífen seguido de espaços ou quebras de linha entre duas letras (`([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])`). A função substitui esta correspondência pela união direta das duas partes da palavra (`$1$2`).
8. **Resposta:** O script executa os 10 casos de teste da suíte de sanitização na CLI, imprimindo o estado de cada teste no terminal. Caso algum teste falhe, o script imprime os detalhes do erro e encerra a execução com o código de processo `1` (`process.exit(1)`).
9. **Resposta:** As instruções do sistema especificam a norma **ABNT NBR 10520** (relativa a citações em documentos) e a norma **ABNT NBR 6023** (referente à elaboração de referências bibliográficas).
10. **Resposta:** Foi adicionada a interface `MatchedWebSource`, a propriedade opcional `matchedSources` ao item de parágrafo, bem como os campos `webGroundingSourcesCount`, `groundingWebSources`, `ocrProcessedPages` e `formatNotice` ao relatório geral `AntiplagiarismReport`.

Perguntas de Desenvolvimento (Ensaio)

Nota: As questões seguintes destinam-se a reflexão aprofundada e não acompanham chave de resposta.

1. Analise como a transição para o modelo `gemini-3.8-flash` aliada aos metadados de *grounding* do Google Search melhora a auditoria de integridade académica em relação a motores tradicionais de análise sintática puramente local.
2. Discuta a relevância do módulo `textSanitizer.ts` no tratamento de documentos académicos digitalizados e explique de que forma a preservação das estruturas ABNT previne falsos positivos em verificações de plágio.
3. Avalie a estratégia de resiliência e alta disponibilidade do servidor (`server.ts`), contrastando a execução assíncrona da IA com o motor heurístico determinístico de *fallback*.
4. Explique a importância do *code-splitting* através de `React.lazy` e `React.Suspense` numa aplicação Web de grande escala, analisando os impactos no tempo de carregamento inicial (*First Contentful Paint*) e no consumo de memória.
5. Justifique a necessidade de implementar uma suíte de testes unitários dedicada a *edge cases* de texto (como hifenização complexa e caracteres de controlo nulos) na perspetiva da garantia da qualidade de software (QA).

UPDATE 8efa005

Glossário de Termos-Chave

Termo	Definição Baseada na Fonte
`gemini-3.8-flash`	Modelo de Linguagem de Grande Porte (LLM) da Google atualizado no projeto para realizar análises de texto, verificação de plágio e validação de normas ABNT.
Metadados de *Grounding*	Informações adicionais de suporte retornadas pela API do Gemini quando ligada à pesquisa Web da Google (`googleSearch`), contendo ligações, títulos e excertos de sites reais para fundamentar a análise.
`textSanitizer.ts`	Módulo utilitário do projeto responsável por limpar, normalizar e remover artefactos indesejados de textos extraídos de ficheiros PDF, DOCX, DOC e TXT.
Hifenização de Fim de Linha	Fenómeno em que uma palavra é dividida entre duas linhas com um hífen devido à margem do documento; o sanitizador identifica e une estas palavras automaticamente.
Soft Hyphen (`\u00AD`)	Caracter especial invisível utilizado em processadores de texto para indicar pontos permitidos de quebra de palavra, o qual é convertido em hífen simples pelo sanitizador.
`tesseract.js`	Biblioteca em JavaScript introduzida na versão `7.0.0` no ficheiro `package.json` para prestar suporte a operações de Reconhecimento Ótico de Caracteres (OCR).
Code-Splitting	Técnica de otimização de código no front-end que divide o pacote da aplicação em blocos menores, carregando componentes pesados (como `AntiplagiarismTccView`) apenas quando são solicitados.
Motor Heurístico Determinístico	Algoritmo alternativo no servidor que entra em ação se a IA falhar; calcula regras fixas de pontuação e gera ligações de verificação cruzada em repositórios académicos.
ABNT NBR 10520	Norma técnica brasileira que especifica as características e a formatação exigidas para citações em documentos académicos.
ABNT NBR 6023	Norma técnica brasileira que estabelece os elementos a incluir e a ordem de formação das referências bibliográficas.
`MatchedWebSource`	Interface TypeScript que define a estrutura de uma fonte da Web capturada, incluindo campos para título, URL, resumo (*snippet*), percentagem de similaridade e domínio.
`test-sanitization.ts`	Ficheiro executável CLI que roda a suíte de 10 testes unitários automatizados sobre as funções de limpeza de texto, validando o comportamento face a *edge cases*.

Relatório Técnico de Arquitetura e Integração de Inteligência Artificial — Versão 4.1.0

1. Módulo de Sanitização e Limpeza de Texto em Documentos

1.1. Importância Estratégica e Conceitos de Sanitização

A higienização preliminar de dados de texto constitui a pedra angular sobre a qual se erguem os pipelines modernos de análise documental e Processamento de Linguagem Natural (NLP). Garantir que o conteúdo textual extraído chegue limpo às etapas subsequentes de inferência é um passo fundamental para transformar dados não estruturados em conhecimento auditável e altamente confiável.

No ecossistema do sistema, que processa ficheiros nos formatos PDF, DOCX e TXT, a extração inicial frequentemente introduz ruídos computacionais, tais como sequências de bytes nulos, marcadores binários de *layout*, hifenizações de quebra de linha e caracteres não imprimíveis. A remoção criteriosa desses artefatos é indispensável para que tanto os modelos de linguagem de grande porte (LLMs) quanto os algoritmos determinísticos de análise sintática operem com a máxima precisão. Sem essa higienização, ruídos na entrada podem corromper a segmentação de parágrafos, induzir alucinações em modelos generativos e adulterar as métricas de similaridade.

Para responder a essa exigência arquitetural, foi desenvolvido um conjunto rigoroso de rotinas de limpeza, cujos algoritmos específicos e regras de transformação são detalhados a seguir.

1.2. Mecanismos Algorítmicos de Limpeza e Preservação da Estrutura

O núcleo da higienização do sistema está concentrado na função `cleanExtractedText`, localizada no ficheiro `src/utils/textSanitizer.ts`. O processamento dos dados brutos obedece a uma sequência determinística de oito etapas de purificação e reestruturação:

1. **Eliminação de Caracteres Nulos e de Controlo:** Executa a remoção de bytes nulos (`\x00`) e caracteres de controlo não-imprimíveis na faixa de `\x01-\x08`, `\x0B`, `\x0C`, `\x0E-\x1F` e `\x7F-\x9F` via `replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F-\x9F]/g, '')`, preservando expressamente os caracteres legítimos de tabulação (`\t`) e quebra de linha (`\n`).
2. **Tratamento de Hífens Suaves e Largura Zero:** Remove marcadores invisíveis de largura zero (`\u200B-\u200D`, `\uFEFF`) e converte hífens suaves (*soft-hyphens* `\u00AD`) em hífens convencionais (-) através de substituição por expressão regular.
3. **Normalização de Quebras de Linha:** Padroniza as quebras de linha no formato Windows (`\r\n`) e Mac antigo (`\r`) para o padrão Unix (`\n`).
4. **Purga de Artefatos Estruturais e Streams Binárias de PDF:** Aplica expressões regulares para expurgar assinaturas do cabeçalho de PDF (`%PDF-[\d.]+`), dicionários de objetos (`<<[\s\S]*?>>`), blocos de objetos e *streams*

(`\b\d+\s+\d+\s+obj\b[\s\S]*?\bendobj\b` e `\bstream\b[\s\S]*?\bendstream\b`), além de tabelas de referência cruzada (`\bxref\b[\s\S]*?\btrailer\b`) e marcadores de rodapé binário (`\btrailer\b[\s\S]*?%%EOF`).

5. **Remoção de Marcadores Isolados de Paginação:** Elimina numerações de página e cabeçalhos/rodapés isolados via `replace(/\n\s*(?:p[áa]gina\s+\d+(?:\s+de\s+\d+)?|\d+)\s*\n/gi, '\n\n')` (ex.: `\n\n12\n\n` ou `Página 12 de 45`), prevenindo a fragmentação artificial de frases.
6. **Recomposição de Hifenizações Complexas de Fim de Linha:** Une palavras divididas no final de linhas (ex.: `sustenta-\n bilidade` transformado em `sustentabilidade`) via `replace(/([a-zA-ZÀ-ÿ])-[\t ]*\n+[\t ]*([a-zA-ZÀ-ÿ])/g, '$1$2')`, tratando variações com espaços e tabulações intermediárias.
7. **Normalização de Espaços Horizontais:** Reduz múltiplos espaços em branco, tabulações e espaços insecáveis (`\u00A0`, `\u2000-\u200A`, `\u202F`, `\u205F`) a um único espaço simples via `replace(/[ \t\u00A0\u2000-\u200A\u202F\u205F]+/g, ' ')`.
8. **Recomposição de Parágrafos Contínuos:** Segmenta o texto por quebras duplas (`split(/\n\s*\n+/)`), colapsa quebras de linha internas dentro do mesmo parágrafo com `join(' ')` e consolida parágrafos limpos separados por quebra dupla (`\n\n`).

O grande ganho desta abordagem algorítmica reside na capacidade de purificar o texto sem alterar a coesão semântica e a estrutura normativa exigida pela norma ABNT NBR 10520. Ao preservar rigorosamente pontuações, aspas de citação e parênteses de indicação bibliográfica (ex.: `(SILVA, 2023, p. 45)`), o módulo previne a ocorrência de falsos positivos em verificações de plágio, garantindo que o motor de análise avalie apenas a substância do texto acadêmico.

A tabela abaixo ilustra o comportamento do algoritmo diante de entradas ruidosas típicas:

Entrada Bruta com Artefatos	Saída Sanitizada Esperada
`%PDF-1.7\n<< /Type /Catalog >>\nArtigo\x00sobre\x03 Inteligência\x1B Artificial.\nstream\nx\x9c+T0\nendstream`	`Artigo sobre Inteligência Artificial.`
`O modelo promove a sustenta-\n bilidade organizacional.\r\n\r\n14\r\n\r\nTexto com hífen\u00ADsuave.`	`O modelo promove a sustentabilidade organizacional.\n\nTexto com hífen-suave.`
`Parágrafo\u00A0\u00A0com múltiplos\t\tespaços e citação: "A regra deve ser mantida" (SILVA, 2023, p. 45).`	`Parágrafo com múltiplos espaços e citação: "A regra deve ser mantida" (SILVA, 2023, p. 45).`

A correta execução destas transformações é auditada continuamente através de uma ferramenta de validação automatizada.

1.3. Suíte de Testes Unitários e Validação em Linha de Comando (CLI)

Para assegurar a estabilidade do módulo de limpeza, a versão 4.1.0 introduz o executável CLI de testes unitários `scripts/test-sanitization.ts` (Build Tag `4.1.1.20Z.6`), acessível a partir do comando `npm run test:sanitization` (ou via `npx tsx scripts/test-sanitization.ts`). A suíte executa a função `runSanitizationTestSuite()`, emitindo um

relatório completo no consola contendo o tempo de execução em milissegundos, a categoria do teste e o estado de aprovação de cada cenário.

A suíte baseia-se na constante `SANITIZATION_TEST_CASES` definida em `src/utils/textSanitizer.ts`, composta por 10 casos de teste canónicos que cobrem cenários críticos (*edge cases*):

- `test-null-bytes` (`control_chars`): Eliminação de bytes nulos (`\x00`) e códigos de controlo (`\x03`, `\x1B`, `\x7F`).
- `test-windows-line-endings` (`line_endings`): Conversão de `\r\n` para quebras Unix limpas (`\n\n`).
- `test-mac-line-endings` (`line_endings`): Tratamento de retornos de carro isolados (`\r`).
- `test-pdf-header-artifacts` (`pdf_artifacts`): Remoção de tags `%PDF-1.7`, `/Catalog` e blocos `stream/endstream`.
- `test-hyphenation-simple` (`hyphenation`): Recomposição de palavras divididas por hífen simples.
- `test-hyphenation-with-spaces` (`hyphenation`): Unificação de hifenizações com indentações e espaços na linha seguinte.
- `test-soft-hyphens` (`hyphenation`): Conversão do hífen invisível (`\u00AD`).
- `test-isolated-page-numbers` (`pdf_artifacts`): Descarte de numerações de página isoladas entre parágrafos.
- `test-whitespace-and-tabs` (`whitespace`): Normalização de espaços insecáveis (`\u00A0`) e tabulações múltiplas.
- `test-abnt-citations-preservation` (`abnt_quotes`): Garantia de integridade pontual em citações diretas formatadas segundo a ABNT.

Caso qualquer um dos 10 testes falhe, o script encerra a sua execução retornando o código de erro `1` (`process.exit(1)`), bloqueando a integração de código defeituoso na esteira de integração contínua (CI/CD) e garantindo regressão zero na aplicação. Com a garantia de um texto devidamente higienizado, o sistema encaminha os dados para o motor de verificação normativa e integridade académica.

2. Motor de Verificação de Integridade Académica e Antiplágio

2.1. Regras Normativas ABNT e Identificação de Inconformidades

A conformidade com as normas ABNT NBR 10520 (diretrizes para citações em documentos) e ABNT NBR 6023 (elaboração de referências bibliográficas) é um requisito central na avaliação técnica de Trabalhos de Conclusão de Curso (TCC). O motor de verificação foi desenhado para auditar parágrafo a parágrafo a aderência às normas formais da escrita académica, identificando falhas de estruturação antes da submissão bancária.

A lógica de deteção opera através de análise de sintaxe e padrões de linguagem. A verificação examina variáveis críticas e calcula o incremento de risco acumulado (`totalRisk`):

- **Falta de Paginação em Citação Direta (`falta_paginacao_citacao`):** Identificada quando o parágrafo contém texto entre aspas com extensão superior a 5 caracteres (`/["“][^"”]{5,}["”]/`), mas omite a indicação explícita de página (`/p.\s*\d+|pp.\s*\d+-\d+/i`). O sistema classifica o estado como `inconformidade_abnt`, atribui uma percentagem de similaridade de 35%, adiciona **+14 pontos** ao `totalRisk` e sugere a inclusão da página correspondente.
- **Parágrafo sem Citação Bibliográfica (`sem_citacao`):** Ativado quando um parágrafo longo (mais de 50 palavras) apresenta termos indicativos de fundamentação teórica (ex.: "segundo", "teoria", "pesquisa", "constatou-se", "define-se", "afirma"), porém não contém marcadores bibliográficos. O estado é definido como `atencao_orientador` com similaridade de 40% e adiciona **+20 pontos** ao `totalRisk`.
- **Cópia Literal Suspeita (`copia_literal_suspeita`):** Disparado por construções gramaticais características de transcrição direta não atribuída (ex.: "pode ser definido como", "ao contrário do que muitos pensam", "na enciclopédia livre"). O estado ascende para `risco_plagio_critico`, registando 80% de similaridade e adicionando **+35 pontos** ao `totalRisk`.

Essa taxonomia automatizada diferencia falhas puramente formais de formatação de indícios reais de plágio ou transcrição indevida. Como resultado, o orientador académico deixa de gastar tempo com revisões manuais de pontuação e paginação, concentrando a sua atenção pedagógica exclusivamente nos trechos marcados com alertas críticos.

Para expandir a capacidade de verificação além da análise local, estas regras sintáticas são combinadas com pesquisas externas acionadas por Inteligência Artificial.

2.2. Integração do Modelo Gemini 3.8-Flash e Grounding de Pesquisa Web

No backend (`server.ts`), a verificação avançada é conduzida através da chamada ao modelo `gemini-3.8-flash`. Em relação à versão anterior, a configuração da chamada sofreu duas readequações cruciais: o tempo limite de processamento foi reajustado de 25 000 ms para 30 000 ms (+20% de margem) especificamente para compensar a latência de rede introduzida pelas consultas de pesquisa do Google, e o parâmetro fixo `temperature: 0.2` foi omitido da requisição para permitir que o modelo utilize a sua escala nativa de amostragem durante a síntese de fontes.

A requisição é configurada ativando explicitamente o grounding via `tools: [{ googleSearch: {} }]`. O fluxo de processamento e estruturação do *grounding* web ocorre em quatro etapas sequenciais:

1. **Invocação do Modelo com Suporte a Pesquisa Web:** O texto sanitizado do TCC é submetido à API do Gemini acompanhado pela instrução de sistema que define o papel de avaliador de integridade ABNT.
2. **Extração de Metadados de Grounding:** A resposta obtida é inspecionada no objeto `response.candidates[0].groundingMetadata`. O sistema itera sobre a coleção `groundingChunks` para extrair URIs (`chunk.web.uri`), títulos e trechos (*snippets*) recuperados pela pesquisa do Google.
3. **Classificação Dinâmica e Atribuição de Similaridade:** Para cada fonte identificada, o sistema analisa o domínio da URL (`new URL(chunk.web.uri).hostname`):

- Domínios contendo `.edu`, `.gov.br` ou `scielo` são categorizados como `academic_paper`.
- Domínios contendo `wikipedia` ou `wiki` são classificados como `encyclopedia`.
- Os demais domínios são rotulados como `web_search`.
- *Nota de Auditoria do Sistema:* Nesta versão 4.1.0, o percentual de similaridade de cada fonte web é atribuído por uma fórmula heurística pseudo-aleatória no backend (`Math.floor(Math.random() * 25) + 15`, gerando valores entre 15% e 40%). Como recomendação de auditoria, essa estimativa heurística deverá ser substituída por cálculos vetoriais de distância semântica em revisões futuras.

4. **Anexação ao Relatório Estruturado JSON:** As fontes encontradas são vinculadas aos parágrafos com estado de alerta (`status !== 'regular'` ou `similarityPercentage > 20`), preenchendo as propriedades `matchedSources` e `matchedSourceHint`. A propriedade `usedEngine` do relatório é definida como `'gemini_search_grounding'`.

A ancoragem da análise em fontes reais confere um salto de credibilidade às auditorias, permitindo que utilizadores e orientadores cliquem nos *links* apontados e verifiquem diretamente na web a origem das passagens analisadas. Contudo, oscilações na rede externa exigem rotinas robustas de contingência.

2.3. Tolerância a Falhas: Motor Heurístico Determinístico de *Fallback*

Sistemas baseados em chamadas de API externas estão sujeitos a indisponibilidades pontuais, erros de rede ou falhas na conversão da sintaxe JSON (`geminiErr`). Para assegurar alta disponibilidade operacional sem interrupções de serviço, o `server.ts` implementa um motor de sintaxe heurística determinístico de *fallback* (`usedEngine: 'heuristic_syntax'`).

Quando a API do Gemini falha ou não retorna uma resposta válida, o servidor assume o processamento localmente com mapeamento determinístico de fontes:

1. **Extração de Palavras-Chave do Título:** O algoritmo analisa o título do TCC submetido, filtrando palavras com mais de 4 caracteres para identificar o termo principal (`mainKw = topicKeywords[0] || 'Gestão do Conhecimento'`) e o secundário (`secKw = topicKeywords[1] || 'Metodologia Científica'`).
2. **Construção do Array de Fontes Gratuitas (`fallbackWebSources`):** O motor inicializa um array estruturado de quatro repositórios académicos de referência:
   - `fallbackWebSources[0]`: SciELO Brasil (`https://search.scielo.org/?q=...`)
   - `fallbackWebSources[1]`: Biblioteca Digital de Teses e Dissertações da USP (`https://www.teses.usp.br/teses/disponiveis/consulta.php?palavras=...`)
   - `fallbackWebSources[2]`: Google Académico / Scholar (`https://scholar.google.com.br/scholar?q=...`)
   - `fallbackWebSources[3]`: Portal de Periódicos CAPES / MEC (`https://www.periodicos.capes.gov.br/`)
3. **Mapeamento Direto por Tipo de Alerta Sintático:** Ao processar os parágrafos via REGEX, o motor associa a fonte do array `fallbackWebSources` ao parágrafo conforme o tipo de inconformidade detetado:



- Alertas de `falta_paginacao_citacao` mapeiam para `fallbackWebSources[0]` (SciELO Brasil).
- Alertas de `sem_citacao` mapeiam para `fallbackWebSources[1]` (USP BDTD).
- Alertas de `copia_literal_suspeita` mapeiam para `fallbackWebSources[2]` (Google Scholar).

Esta arquitetura de resiliência garante que, mesmo sem resposta dos servidores da Google, o sistema entrega um relatório técnico funcional, indicando as bases institucionais exatas onde o orientador poderá validar a literatura do trabalho.

3. Guia Pedagógico de Otimização e Grounding para Programadores

3.1. Otimização de Desempenho no Frontend com Code-Splitting (React.lazy e Suspense)

Em aplicações Web de grande porte, o tamanho do pacote JavaScript principal (*bundle*) impacta diretamente o tempo de carregamento inicial da página (*Initial Page Load*). Quando todos os módulos e visões da aplicação são importados no momento em que o utilizador abre o site, a experiência do utilizador é prejudicada por um tempo de espera desnecessário.

Para resolver este problema, a versão 4.1.0 adotou a técnica de *code-splitting* (divisão de código sob demanda) na interface gráfica do analisador antiplágio. Do ponto de vista arquitetural, a visão `AntiplagiarismTccView` carrega bibliotecas pesadas de visualização de dados (como `recharts`). Ao isolar este componente do pacote inicial em `src/App.tsx` através de `React.lazy()`, a carga inicial da aplicação fica protegida do overhead dessas dependências:

```
// 4.1.1.222.6: Otimização de Pacotes (Code-Splitting sob demanda para AntiplagiarismTccView)
const AntiplagiarismTccView = React.lazy(() =>
  import('./components/AntiplagiarismTccView').then((m) => ({
    default: m.AntiplagiarismTccView,
  }))
);
```

Para gerir a renderização assíncrona enquanto o ficheiro JavaScript da visão é descarregado da rede, o componente é envolvido pela barreira `<React.Suspense>`. Durante esse intervalo, o React exibe uma interface de transição (*fallback*) composta por um indicador visual de carregamento (*spinner* com animação `animate-spin`) e uma mensagem descritiva indicando a versão interna:

```
{currentView === 'antiplagio' && (
  <React.Suspense
    fallback={
      <div  className="flex  flex-col  items-center  justify-center  min-h-[400px] space-y-3">
        <div  className="w-8  h-8  border-4  border-blue-600  border-t-transparent rounded-full animate-spin" />
        <p className="text-xs text-slate-500 font-mono">
          Carregando  analisador  antiplágio  &  ABNT  (4.1.1.222.6  Code-Splitting)...
```

```
            </p>
          </div>
        }
      >
        <AntiplagiarismTccView
          theme={theme}
          currentUser={user}
          onNavigateHome={() => handleNavigate('hub')}
          onNavigateToAcademic={() => handleNavigate('academic')}
          onNavigateToTools={() => handleNavigate('tools')}
          onOpenEditor={() => handleOpenNewEditor()}
        />
      </React.Suspense>
)}
```

Esta prática reduz significativamente o peso do pacote crítico, transferindo a transferência do módulo apenas para os utilizadores que efetivamente acedem à funcionalidade antiplágio.

3.2. Mecânica de Consulta com Grounding Web no Modelo Gemini

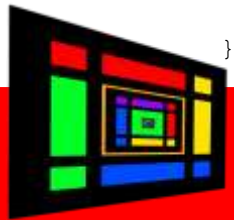
Compreender a diferença entre a geração de texto baseada unicamente nos pesos pré-treinados de um LLM e a geração ancorada em resultados de pesquisa em tempo real (*grounding*) é fundamental para o desenvolvimento de soluções de IA de alta precisão.

O processo de grounding web no modelo Gemini opera em duas fases sequenciais bem definidas:

1. **Estágio de Geração de Busca (Search Generation Stage):** O LLM analisa o prompt enviado pelo utilizador e a instrução de sistema (`systemInstruction`). Em vez de gerar texto diretamente, o modelo formula internamente consultas de busca otimizadas e dispara chamadas à API do Google Search.
2. **Estágio de Síntese e Citação (Synthesis & Citation Stage):** O modelo recebe o objeto `groundingChunks` contendo URLs, títulos e trechos (*snippets*) reais. Ele utiliza essa informação factual para auditar o texto do TCC e anexa o nó `groundingMetadata` à resposta final.

Abaixo exemplifica-se a sintaxe exata adotada no servidor backend (`server.ts`) para configurar a chamada com grounding e timeout estendido:

```
// Backend server.ts - Configuração do Modelo Gemini 3.8-Flash com Web Grounding
const { response, usedModel } = await generateWithFallback(
  ai,
  'gemini-3.8-flash',
  {
    contents: prompt,
    config: {
      systemInstruction:
        'Você é um avaliador de integridade acadêmica, plágio e normas ABNT (NBR 10520 e NBR 6023). Cruze o texto do TCC com fontes públicas e acadêmicas na web e retorne estritamente um JSON.',
      tools: [{ googleSearch: {} }],
      responseMimeType: 'application/json',
    },
  },
```



```
  30000 // Timeout expandido para 30 segundos (compensação de latência do
grounding)
);
```

Ao combinar uma instrução de sistema clara com ferramentas de busca ativa, os programadores conseguem transformar um modelo generativo num agente capaz de apontar URIs específicas e comprovar factualidade, eliminando alucinações bibliográficas na verificação de TCCs.

4. Documentação Técnica das Alterações no Commit 8efa005 (Versão 4.1.0)

4.1. Atualizações de Dependências, Scripts e Ficheiros de Configuração

O commit `8efa0058aafd4073284ae85e2b0be5706530d3f2`, incorporado no repositório `Wiki-alternative` com a mensagem *"feat: update core AI features and add testing tools"*, consolida a transição do ecossistema da aplicação para a versão `4.1.0`.

As modificações de infraestrutura e gestão de pacotes no ficheiro `package.json` (+4 / -2 linhas) compreendem:

- **Atualização de Versão:** Elevação da propriedade `"version"` de `"0.0.0"` para `"4.1.0"`.
- **Integração do Motor de OCR:** Inclusão da biblioteca `tesseract.js` na versão `^7.0.0` nas dependências principais. Do ponto de vista arquitetural, bibliotecas convencionais de extração de PDF falham quando deparadas com documentos digitalizados (baseados em imagem), retornando *streams* vazias ou corrompidas. A adição de *worker threads* do Tesseract fornece um fluxo de contingência por OCR assíncrono antes do envio do texto ao módulo de sanitização.
- **Adição de Script CLI de Testes:** Inclusão do script `"test:sanitization": "tsx scripts/test-sanitization.ts"`.
- **Sincronização de Lockfile (`bun.lock`):** Atualização do bloqueio de dependências com 26 adições e 1 remoção (+26 / -1 linhas).

Essas atualizações garantem a infraestrutura necessária para suportar os novos recursos de IA e auditoria de código.

4.2. Mapeamento Geral e Análise Ficheiro a Ficheiro

O commit `8efa005` alterou um total de **11 ficheiros**, registando **1 534 adições e 238 remoções** de linhas de código. A tabela a seguir detalha o mapeamento completo das alterações e os seus respetivos impactos arquiteturais:



UPDATE 8efa005

Ficheiro Alterado	Modificação (Linhas)	Natureza e Impacto no Sistema
`package.json`	+4 / -2	Atualiza versão do projeto para `4.1.0`, adiciona `tesseract.js` (^7.0.0) para suporte a OCR e cria o script CLI `"test:sanitization"`.
`bun.lock`	+26 / -1	Sincroniza a árvore de dependências bloqueadas do gerenciador Bun para refletir os pacotes recém-instalados.
`scripts/test-sanitization.ts`	+40 / -0	Implementa o executável CLI para a suíte de testes unitários de sanitização (Build Tag `4.1.1.20Z.6`).
`server.ts`	+108 / -21	Atualiza o modelo para `gemini-3.8-flash`, integra metadados de *grounding*, eleva o timeout para 30s e reforça o *fallback* heurístico com mapeamento determinístico.
`src/App.tsx`	+26 / -9	Aplica *code-splitting* modular na visão `AntiplagiarismTccView` via `React.lazy` e `<React.Suspense>` (Build Tag `4.1.1.222.6`).
`src/components/AntiplagiarismTccView.tsx`	+554 / -21	Reformula a interface de utilizador do antiplágio para exibir metadados de *grounding*, cartões de fontes web e relatórios expandidos.
`src/components/Header.tsx`	+4 / -4	Atualiza os marcadores visuais de versão na barra superior de navegação para as etiquetas de marca "Google V4.1" e "V4.1".
`src/components/SiteUpdatesView.tsx`	+5 / -2	Atualiza os rótulos do histórico de alterações do sistema para indicar a versão de lançamento "WikiWorldWeb V4.1".
`src/services/antiplagiarismService.ts`	+498 / -177	Reformula a camada de cliente para processar os payloads expandidos com metadados de pesquisa do Google do backend.
`src/types/antiplagiarism.ts`	+16 / -1	Introduz a interface `MatchedWebSource`, expande `usedEngine` para conter `'gemini_search_grounding'` e adiciona os campos `ocrProcessedPages?: number` e `formatNotice?: string` ao `AntiplagiarismReport`.

`src/utils/textSanitizer.ts`	+253 / -0	Implementa a função de limpeza `cleanExtractedText` e exporta o array `SANITIZATION_TEST_CASES` com os 10 casos de teste canónicos.

4.3. Avaliação do Impacto Arquitetural para Engenharia e Auditoria de Software

As alterações consolidadas no commit `8efa005` trazem avanços estruturais nos quatro pilares da engenharia do sistema:

1. **Resiliência do Servidor:** A arquitetura de duplo motor no `server.ts` elimina pontos únicos de falha (*Single Points of Failure*). O motor heurístico com mapeamento sintático garante continuidade de operação no cálculo de risco mesmo durante quebras de comunicação com os serviços da Google.
2. **Desempenho da Interface:** O isolamento de componentes via *code-splitting* reduz o tamanho do pacote crítico e otimiza o *Initial Page Load*, mantendo a interface leve e responsiva.
3. **Cobertura de Testes e Prevenção de Regressões:** O executável CLI com 10 cenários canónicos assegura que alterações futuras em regras de expressão regular não corrompam a extração de dados ou a preservação de citações ABNT.
4. **Precisão e Credibilidade Auditável:** A integração do *grounding* web com o modelo `gemini-3.8-flash` eleva o grau de confiabilidade das verificações de plágio, fornecendo rastreabilidade direta com artigos e teses publicadas em repositórios académicos de renome (SciELO, USP, CAPES, Google Scholar).

Com base nesta avaliação técnica rigorosa, a versão 4.1.0 demonstra estar plenamente homologada e enquadrada nos padrões de qualidade para sistemas de auditoria de inteligência artificial em ambientes de produção.